ALUMNI CONNECT

PRODUCT REQUIREMENTS DOCUMENT

Alumni mentorship, career guidance and mock interview management

DOCUMENT CONTROL

Field	Value
Document ID	AC-PRD-001
Version	1.0
Status	Project baseline / Draft for team review
Prepared	02 November 2026
Product	AlumniConnect
Source baseline	Alumni–Student Mentorship Platform: DBMS Project Blueprint

Purpose: Define the product scope, user roles, system behaviour and quality expectations for the AlumniConnect DBMS project. This PRD is intended to guide design, implementation, testing and demonstration.

1. Product overview

AlumniConnect is a web-based Alumni–Student Mentorship and Mock Interview Platform. It connects students with alumni who have relevant skills, domain knowledge and industry experience. Students can discover mentors, request mentorship and book sessions. Alumni can manage their professional profiles and availability, respond to requests, conduct offline sessions and record outcomes. Administrators maintain platform reference data, manage users and review platform analytics.

> **Product principle**
>
> The product is more than a mentor directory. It manages the full lifecycle from discovery and request through allocation, scheduling, offline session outcomes and feedback, while demonstrating sound relational database design and specialised data stores.

1.1 Problem statement

Students may find it difficult to identify alumni whose technical skills, professional experience and career domain match their goals. Alumni need a consistent way to manage incoming requests, availability, sessions and student progress. Informal coordination makes it harder to track decisions, prevent scheduling conflicts and evaluate mentorship outcomes.

1.2 Product goals

- Give students a structured way to discover suitable alumni mentors and understand why they are recommended.
- Allow mentors to manage their expertise, availability, requests and session records.
- Make mentorship requests, allocations and bookings traceable and protected by database integrity rules.
- Capture session outcomes, two-way session feedback and student reviews of mentors.
- Provide administrators with reference-data management and useful SQL-based operational reports.
- Demonstrate DBMS concepts through MySQL transactions, Redis caching and Neo4j relationship-based recommendations.

1.3 Success indicators

Outcome	How success is demonstrated
Discovery	A student can retrieve relevant mentor candidates and see the matching reasons.
Allocation integrity	Accepting a request creates no duplicate active allocation and leaves a history of the decision.
Booking integrity	Concurrent attempts to reserve one mentor slot result in at most one confirmed booking.
Session traceability	A confirmed booking can be tracked through a scheduled, completed, cancelled or no-show session outcome.
Feedback integrity	Feedback and reviews are restricted to valid session participants and completed sessions as applicable.
DBMS demonstration	The project demonstrates SQL analytics, transactions/concurrency, Redis cache operations and Neo4j traversal.

1.4 Product scope

Included in core release	Not included in core release
Student, mentor and administrator portals; authentication and role authorization.	Video conferencing or live virtual interview infrastructure.

Included in core release	Not included in core release
Profiles, skills, domains, mentor discovery and Neo4j recommendations.	Vector/semantic search; the blueprint places this in a later phase.
Mentorship requests, allocation, availability, bookings and offline session records.	A separate independently deployed microservice for every business module.
Feedback, mentor reviews, notifications, SQL reports and Redis caching.	Real-money payments, subscriptions or external recruitment workflows.

2. Actors and system context

The application has three primary human roles. A background worker supports reliable notification delivery and synchronisation tasks. The API boundary is the single logical entry point to the backend; it is not a separate human actor.

Actor	Description	Primary goals / permissions
Student / Mentee	A current student seeking career guidance, skills development or mock interview practice.	Maintain student profile, list skills and interests, discover mentors, submit/cancel requests, book valid slots, attend sessions, submit session feedback and review a mentor.
Alumni / Mentor	An alumnus willing to guide students using professional and industry experience.	Maintain professional profile, list skills and domains, publish availability, accept/reject requests, manage mentorship capacity, record offline session outcomes and provide session feedback.
Administrator	A trusted platform operator responsible for configuration and oversight.	Manage users and platform reference data such as departments, skills and mentorship domains; access administrative analytics and reports.
Background worker	An internal process that handles queued events after a database transaction commits.	Deliver event notifications and project committed MySQL changes to Neo4j / invalidate relevant Redis cache entries.

2.1 System context

Component	Responsibility in the product
Student / Mentee client	Student-facing interface for profile, discovery, requests, bookings, sessions and feedback.
Alumni / Mentor client	Mentor-facing interface for professional profile, availability, request decisions and session records.
Administrator portal	Administrative interface for user/reference-data management and analytics.
API boundary / backend	Single logical entry point; one modular backend application containing identity, profile, matching, request, booking, session, feedback and notification modules.
MySQL + InnoDB	Authoritative store for users, profiles, skills, requests, allocations, availability, bookings, sessions, feedback, reviews, notifications and audit data.
Redis	Performance layer for cacheable profiles, reference data, recommendations and short-lived state. It must not decide booking availability.
Neo4j	Graph projection used to traverse student, mentor, skill, domain, company and department relationships for recommendations.

Source-of-truth rule

MySQL owns authoritative business data. Redis is a cache / temporary-state layer. Neo4j is a relationship-optimised projection. A recommendation from Neo4j is only a candidate: MySQL must validate current eligibility, capacity and booking rules.

2.2 Primary user journeys

- Student journey: create account → complete profile, skills and domain interests → discover/review mentors → submit a mentorship request → receive a decision → book an available slot → attend an offline session → review feedback and progress.
- Mentor journey: create account → complete professional profile, skills and expertise → publish availability and capacity → review requests → accept or reject → conduct the offline session → record outcomes and feedback.
- Administrator journey: manage user access and platform reference data → inspect reports on requests, allocations, sessions, ratings and platform activity.

3. Functional requirements

Requirement priority uses MUST for core-release behaviour and SHOULD for a capability that materially improves traceability or administration but may be staged if implementation time is constrained. Each acceptance statement is intended to be testable.

3.1 Identity, access and profiles

FR-01	**Account registration and sign-in** [MUST] The system shall allow a person to register and sign in using an account identity. A common User identity shall be associated with role-specific Student and/or Mentor profile information, so authentication data is not duplicated across profiles. **Acceptance:** A registered user can authenticate; invalid credentials are rejected; the authenticated identity is consistently linked to the permitted profile(s).
FR-02	**Role-based authorization** [MUST] The system shall distinguish Student/Mentee, Alumni/Mentor and Administrator roles. Protected operations shall be available only to users whose roles and ownership permissions allow them. **Acceptance:** A student cannot edit another user's profile or make mentor-only decisions; a mentor cannot access administrator controls; administrative operations require administrator authorization.
FR-03	**Student profile management** [MUST] A student shall be able to create, view and update their student profile, including relevant academic / department information, skills and mentorship interests. **Acceptance:** The student can save changes and see the updated profile; skill and domain associations are represented without duplicating skill or domain master records.
FR-04	**Mentor profile management** [MUST] An alumni mentor shall be able to maintain a professional profile containing relevant experience, company, role, skills and mentorship-domain expertise. **Acceptance:** The mentor can update professional information; changes are reflected in the authoritative MySQL record and eventually in the recommendation projection / caches.

FR-05	**Skills and domain catalogue** [MUST] The system shall maintain reusable Skill, Department, Company and Mentorship Domain reference records. Administrators shall be able to maintain the managed catalogues. **Acceptance:** A skill or domain can be associated with multiple profiles; the same catalogue value is not redundantly created for each user.
FR-06	**Student skills and interests** [MUST] A student shall be able to associate multiple skills and preferred mentorship domains with their profile. **Acceptance:** A student can add/remove skills and interests; the application presents the current saved associations and does not create duplicate associations.
FR-07	**Mentor expertise and capacity** [MUST] A mentor shall be able to associate multiple skills and domains with their profile and maintain the information used to determine whether they can accept another mentee. **Acceptance:** Mentor expertise is available to discovery and recommendation; the current capacity value is checked against MySQL before allocation or booking is confirmed.

4. Functional requirements

4.1 Mentor discovery, recommendations and allocation

FR-08	**Mentor directory and discovery** [MUST] The student client shall provide a way to browse or search mentor profiles using relevant profile information such as skills, mentorship domains, company and professional experience. Private information shall not be exposed through the public-facing directory. **Acceptance:** A student can retrieve matching mentor cards and view the mentor information permitted for discovery; private account data is not included in directory results.
FR-09	**Graph-based mentor recommendations** [MUST] The platform shall use Neo4j relationship traversal to identify potentially relevant mentors. Recommendation dimensions shall include skill similarity, domain relevance, relevant experience, career relevance, mentor quality and capacity. The relative weights shall be configurable rather than permanently hard-coded. **Acceptance:** For a student profile, the platform returns candidate mentors and scores / matching dimensions; candidates who fail current MySQL eligibility or capacity checks are excluded from final results.
FR-10	**Recommendation explanation and history** [SHOULD] The system should explain why a mentor is recommended, for example shared skills, matching domain, relevant experience or prior mentoring outcomes. It should retain recommendation runs and candidate results so changes over time can be analysed. **Acceptance:** A student can inspect the reasons for a recommendation; an administrator or authorised report can compare saved recommendation runs, scores and subsequent selection / mentorship outcomes.
FR-11	**Mentorship request submission** [MUST] A student shall be able to submit a mentorship request to a selected mentor. The request shall be tracked with a controlled lifecycle: Pending, Accepted, Rejected or Cancelled. **Acceptance:** A request appears in the student's history and the mentor's pending requests; duplicate submissions are handled consistently; cancelled or rejected requests remain available for history and reporting.

FR-12	**Mentor decision and allocation** [MUST] A mentor shall be able to accept or reject a pending request. Before acceptance, the system shall verify that the request is still pending, the mentor can accept another allocation and no duplicate active allocation exists. An accepted request shall create a Mentor Allocation. **Acceptance:** Accepting a request changes request status and creates at most one allocation in one MySQL transaction; a rejected request is retained in history; failed validation leaves no partial allocation.

4.2 Scheduling and booking

FR-13	**Mentor availability management** [MUST] A mentor shall be able to publish and maintain availability slots and preferred timings for mentorship or mock interview sessions. **Acceptance:** A student can view slots that are currently eligible for booking; a slot that has already been claimed is not presented as available based solely on cached data.
FR-14	**Atomic booking** [MUST] A student shall be able to book an eligible mentor slot for a mentorship or mock interview. Before confirmation, the system shall validate slot availability, student-side overlapping appointments, mentor capacity, a valid request/allocation where required and duplicate submissions. **Acceptance:** Under concurrent requests for the same slot, at most one booking succeeds; competing requests receive a clear unavailable/conflict response; database rollback leaves no partial booking.
FR-15	**Booking cancellation and history** [MUST] The system shall allow an authorised participant or administrator to cancel a booking according to the project's configured policy. Booking status changes and their history shall be retained. **Acceptance:** A cancellation is persisted, the slot is recalculated from authoritative booking state and both participants are eligible for a cancellation notification; the record is not physically erased.

5. Functional requirements

5.1 Offline session management

FR-16	**Session creation and outcome recording** [MUST] A confirmed booking may produce a Mentorship Session record. The platform shall support offline meeting tracking; it does not need video-conferencing integration for the core release. After the meeting, the mentor shall record whether it was completed, cancelled or marked no-show, discussion topics, skills assessed, performance observations, areas for improvement, next steps and recommended preparation. **Acceptance:** The session follows Scheduled → Completed / Cancelled / No-show; the mentor can save the outcome; session history remains available to authorised participants.
FR-17	**Session feedback** [MUST] The system shall support structured session feedback from either actual participant. Feedback may cover communication, helpfulness, technical preparation and session notes. **Acceptance:** Feedback cannot be submitted before the session has taken place; only the actual participants can submit it; the feedback is linked to the correct session.

FR-18	**Mentor review and rating** [MUST] The student shall be able to submit a mentor review after a completed session. A mentor review is distinct from general session feedback. **Acceptance:** A review references a valid completed session; one student can submit at most one mentor review per session; review history is not silently overwritten.

5.2 Notifications and operations

FR-19	**Event-driven notifications** [MUST] The system shall generate notifications for key events: request submitted (mentor), request accepted/rejected (student), booking confirmed/cancelled (both participants), session reminder (both participants) and feedback submitted (relevant participant). **Acceptance:** Each committed event creates the corresponding notification intent; an operation that rolls back does not result in a delivered notification; notification processing can be retried safely.
FR-20	**Transactional outbox processing** [MUST] The platform shall record notification and projection events in a MySQL transactional outbox within the same business transaction. A background worker shall process committed outbox events to deliver notifications, update Neo4j projections and invalidate relevant Redis cache entries. **Acceptance:** If the business transaction rolls back, no outbox event is committed; if a worker retries an event, duplicate side effects are prevented or safely handled.
FR-21	**Administration of users and reference data** [MUST] Administrators shall be able to manage users and the reference data required by the platform, including departments, skills and mentorship domains. **Acceptance:** An administrator can perform permitted management actions; an unauthorised user cannot invoke them; changes are auditable.
FR-22	**Operational analytics and reporting** [MUST] The administrator shall be able to access SQL-backed reports for platform oversight. Useful reports include request-to-allocation-to-session funnel, mentor ratings, completed sessions, mentors by domain and students without an allocation. **Acceptance:** Reports are derived from authoritative data and return consistent counts / aggregates for the selected criteria; private fields are excluded unless required by an authorised administrative task.
FR-23	**Redis cache management** [MUST] The system shall use Redis for rebuildable performance data such as frequently viewed mentor profiles, domain/skill reference data and recent recommendation results. It may also store short-lived login challenges, rate-limit counters and idempotency records. Cache-aside, TTL and invalidation shall be used where appropriate. **Acceptance:** The system demonstrates cache hit, miss and expiry; profile changes invalidate relevant cache entries; a cache hit never bypasses MySQL authorization or booking validation.

6. Business rules and data requirements

6.1 Mandatory business rules

Rule ID	Business rule
BR-01	A user has one common authentication identity. Student and Mentor information is held in separate role-specific profile concepts.
BR-02	A student can have multiple skills; a mentor can have multiple skills. These many-to-many associations must be represented explicitly.
BR-03	A mentorship request has one controlled status: Pending, Accepted, Rejected or Cancelled. Rejected and cancelled requests remain in history.
BR-04	A request may result in at most one allocation. Acceptance validates current request state, mentor capacity and duplicate active allocations.
BR-05	A booking is a planned appointment; a Mentorship Session is the actual meeting and its outcome. These are separate concepts.
BR-06	A specific mentor availability slot can be booked at most once. Student-side overlapping appointments must also be prevented.
BR-07	MySQL is authoritative for availability, bookings, allocation, session status, ratings and feedback. Redis must not make the final booking decision.
BR-08	Neo4j recommendations are candidate suggestions. Current eligibility and capacity are checked against MySQL before action.
BR-09	Neo4j and Redis are synchronised from committed MySQL events. Neo4j may be eventually consistent; MySQL wins if records disagree.
BR-10	Session feedback is available only after the session has taken place and only to actual participants.
BR-11	A student may submit at most one mentor review per completed session. Review history must be preserved.
BR-12	Notification intents are committed through the outbox in the same transaction as the business change that generated them.

6.2 Core conceptual data areas

Data area	Main concepts
Identity and profiles	User, Role, Student Profile, Mentor/Alumni Profile, Department, Company.
Skills and domains	Skill, Student Skill, Mentor Skill, Mentorship Domain, Student Domain Interest, Mentor Domain Expertise.
Matching and allocation	Mentorship Request, Recommendation Run, Recommended Mentor, Mentor Allocation.
Scheduling and session	Mentor Availability, Booking, Mentorship Session.
Feedback and communication	Session Feedback, Mentor Review, Notification, Notification Outbox, Audit Log.

6.3 Relationship expectations

Relationship	Expected cardinality / meaning
User → Student Profile	1 : 0..1; an account may have a student profile.
User → Mentor Profile	1 : 0..1; an account may have a mentor profile.
Student ↔ Skill / Mentor ↔ Skill	Many-to-many; use explicit association concepts.
Student → Mentorship Request	1 : N; a student can submit multiple requests.
Request → Allocation	1 : 0..1; a request creates at most one allocation.

Relationship	Expected cardinality / meaning
Allocation → Session	1 : N; an allocation can lead to multiple sessions.
Mentor → Availability	1 : N; a mentor can publish multiple slots.
Availability → Booking	1 : 0..1; a slot is booked at most once.
Booking → Session	1 : 0..1; a confirmed booking may create a session record.

6.4 Data authority and synchronisation

- MySQL is the system of record for identity, profiles, skills, requests, allocations, availability, bookings, sessions, ratings, feedback, notifications and audit records.
- Redis stores temporary or rebuildable data for speed. Use TTL and explicit invalidation for profile and recommendation caches. Never trust it as the authority for booking availability.
- Neo4j stores a graph representation of student, mentor, skill, domain, company and department relationships. It is a recommendation projection, not the authoritative store for profile or booking data.
- On a relevant profile or relationship change: commit MySQL data and an outbox event together; process the event asynchronously; update Neo4j; invalidate affected Redis keys.

7. Non-functional requirements

> **Interpretation of this section**
>
> The project blueprint specifies architecture and business safeguards but does not set numeric service-level targets. The measurable thresholds below are proposed initial acceptance targets for this academic project. They should be confirmed against the team's deployment environment and test data before final sign-off.

NFR-01	**Security and authorization** [MUST] All protected operations shall enforce authorization on the backend, not only by hiding interface controls. Passwords shall be stored using secure one-way salted hashing; secrets shall not be committed to source control. **Acceptance:** Role and ownership tests cover student, mentor and administrator actions; unauthorised API calls are rejected; plaintext passwords and secrets are not present in the database or logs.
NFR-02	**Privacy and data exposure** [MUST] The platform shall expose only the profile attributes required for mentor discovery to users who are authorised to view them. Administrative, authentication and private account data shall not be included in public directory responses. **Acceptance:** A directory/API response test confirms that private fields are omitted; access to private records is restricted to the owner or authorised administrator.
NFR-03	**Transactional integrity** [MUST] Allocation, booking, cancellation and event creation shall preserve consistency through database transactions and integrity constraints. Concurrent booking requests shall not create duplicate slot reservations or overlapping student appointments. **Acceptance:** Automated concurrent tests show one successful reservation per slot; injected failures roll back all related changes; uniqueness and foreign-key constraints reject invalid references.

NFR-04	**Cross-database consistency** [MUST] MySQL shall remain authoritative. Redis and Neo4j updates shall be driven from committed outbox events and be recoverable through retries/rebuilds. Stale cached or graph data shall not override current MySQL validation. **Acceptance:** A committed profile change eventually updates the graph projection and invalidates related cache entries; a rolled-back change is not projected; booking remains correct if Redis or Neo4j is stale or unavailable.
NFR-05	**Performance - proposed baseline** [SHOULD] For an initial test profile of up to 50 concurrent users, common read/API operations should complete within 2.5 seconds at the 95th percentile; mentor recommendations within 4 seconds; booking confirmation within 3 seconds, excluding external network delays. **Acceptance:** The team executes a repeatable load test and records p50/p95 latency, throughput and error rate; results are documented with dataset size and environment. These are project targets, not guarantees from the blueprint.
NFR-06	**Scalability and modularity** [SHOULD] The backend shall be organised as a modular monolith with clearly separated business modules and a shared primary relational database. Modules should be separable in code without requiring independent microservice deployment. **Acceptance:** Identity, profile, matching, request, booking, session, feedback and notification logic have clear module boundaries and can be tested independently.

8. Non-functional requirements

NFR-07	**Reliability and graceful failure** [MUST] A failed database operation shall not leave partial business state. Temporary failures in notification delivery or graph/cache projection shall be retryable without losing the committed business transaction. **Acceptance:** Failure injection tests show transaction rollback and safe worker retry; users receive a clear result when an operation cannot be completed.
NFR-08	**Auditability and history** [MUST] Important state changes shall be traceable through audit/history data, including request decisions, allocation, booking changes, session outcomes and administrative changes. Historical records shall not be silently erased or overwritten. **Acceptance:** An authorised reviewer can trace key lifecycle changes to actor, time and target record; review and request history remains available after status changes.
NFR-09	**Usability and responsive layout** [SHOULD] Student, mentor and administrator workflows shall use role-appropriate navigation and clear validation/error messages. The web interface should remain usable on common desktop, tablet and mobile viewport sizes. **Acceptance:** Representative workflows can be completed without relying on hidden or role-inappropriate controls; key pages render without horizontal overflow at the agreed viewport test sizes.
NFR-10	**Accessibility** [SHOULD] Forms, navigation, status indicators and interactive controls should support keyboard use, visible focus and meaningful labels. Colour shall not be the only indicator of status. **Acceptance:** A manual accessibility review verifies keyboard access, labels, focus visibility and readable contrast for the main workflows.

NFR-11	**Maintainability and testability** [MUST] The application shall separate business logic from database access and interface concerns sufficiently to support unit, integration and end-to-end testing. Database migrations and environment configuration shall be repeatable. **Acceptance:** A clean development setup can create the schema and run automated tests; business rules are covered by tests; configuration secrets are provided outside source code.
NFR-12	**Backup and recovery** [SHOULD] The team shall maintain a repeatable way to back up and restore the authoritative MySQL database for demonstration and development recovery. Redis and Neo4j projections shall be rebuildable from MySQL where required. **Acceptance:** A restore exercise can recover core MySQL records; cache and graph data can be repopulated or synchronised from authoritative data.
NFR-13	**Compatibility** [SHOULD] The web clients should support current stable versions of major desktop browsers used by the project team and provide a responsive layout for mobile browsers. **Acceptance:** The team records a browser smoke-test matrix for the agreed browsers and viewport sizes.
NFR-14	**Observability and safe logging** [SHOULD] The backend and worker should emit structured application logs for errors, transaction outcomes and background event processing. Logs shall not contain passwords, tokens or unnecessary sensitive profile data. **Acceptance:** A failed booking or outbox delivery can be diagnosed from logs using correlation identifiers without exposing credentials or private data.

8.1 Operational constraints

- The primary relational database is MySQL 8.x with InnoDB; it is the final authority for transactional business rules.
- Redis is used for caching and short-lived state. A cache outage must not make incorrect bookings possible.
- Neo4j is used for relationship-based discovery. MySQL validates capacity, identity and final eligibility.
- The backend is a modular monolith with one logical API boundary; independent microservice deployments are not required.
- Notification delivery and cross-database projection are asynchronous worker responsibilities backed by a transactional outbox.

9. Release boundaries, acceptance and open decisions

9.1 Core release scope

Core release capability	Acceptance evidence
Student, mentor and administrator portals	Each role can sign in and access only its permitted workflows.
Authentication and role-based authorization	Positive and negative authorization tests pass for all protected operations.
Profiles, skills and mentorship domains	Student and mentor data can be maintained and correctly associated with shared reference records.
Neo4j recommendations	A student receives relevant candidates with reasons; MySQL eligibility checks filter unavailable / ineligible mentors.

Core release capability	Acceptance evidence
Requests and allocation	Request state transitions work; acceptance and allocation occur atomically without duplicate active allocations.
Availability and booking	Mentors publish slots; atomic concurrency test proves a slot cannot be double-booked and students cannot have overlapping appointments.
Offline sessions	Mentor records scheduled/completed/cancelled/no-show outcome and structured session details.
Feedback and ratings	Only actual participants can submit feedback; one review per student per completed session is enforced.
Notifications	Outbox-backed notifications are produced for the defined events and survive safe retry.
SQL analytics and Redis caching	Project demonstrates meaningful reports plus cache hit, miss, expiry and invalidation.

9.2 Deferred / out of scope

- Vector or semantic mentor search. The blueprint positions this as a later phase after SQL and graph recommendations are working.
- Video conferencing or live interview streaming. Core session tracking is for offline meetings and post-session recording.
- Independent microservice deployment for each module. The recommended architecture is a modular monolith.
- Payments, subscriptions, recruitment/placement workflows or real-money transactions; these are not specified in the project blueprint.

9.3 Assumptions

- Users access the system through role-appropriate web clients and the central backend API.
- A mentor is permitted to accept requests only while below a configured mentorship capacity.
- A confirmed booking represents a planned offline appointment; session outcome is recorded separately after the meeting.
- The primary database can enforce foreign keys, uniqueness rules and transactional locking required for core integrity.
- Redis and Neo4j can be rebuilt or resynchronised because their data is derived from authoritative MySQL data.

9.4 Decisions to confirm before implementation

Decision	Why it must be confirmed
Account eligibility and onboarding	Who may register as a student or alumnus, and how is alumni status verified? The blueprint specifies roles but not a verification process.
Recommendation weights and thresholds	The blueprint requires configurable matching dimensions but does not prescribe numeric weights or minimum scores.
Mentor capacity policy	Define default capacity, whether capacity is global or domain-specific, and when it is decremented / released.
Slot duration, timezone and booking rules	Define default session lengths, minimum notice, rescheduling and cancellation windows.
Notification channels	Choose in-app, email or other delivery channels; the blueprint defines events and outbox behaviour, not delivery providers.
Review scale and moderation	Choose the rating scale, review visibility and any moderation / edit policy.

Decision	Why it must be confirmed
Administrator permissions	Define whether admins can suspend users, edit profiles, cancel bookings or only manage reference data and reports.
Numeric NFR targets	Confirm proposed latency and concurrency targets against the actual development and demo environment.

10. Traceability to the DBMS project

AlumniConnect is intended to demonstrate the DBMS concepts covered across the course units. This traceability keeps the application requirements connected to the academic project objectives without making the product depend on academic terminology at runtime.

DBMS area	AlumniConnect application evidence
Unit 1 - relational modelling	Domain entities, ER design, relational mapping, keys, foreign keys, many-to-many junctions and integrity constraints.
Unit 2 - advanced SQL	Joins for mentor cards; nested/correlated subqueries for mentor and allocation reports; views for safe public directory; triggers/audit; stored procedures/functions; CTE funnel analysis; window-function mentor ranking; full-text profile search.
Unit 3 - normalisation and optimisation	Functional dependencies, candidate keys, partial/transitive dependency analysis, justified 3NF/BCNF design, indexes and EXPLAIN-based plan comparisons for discovery, history and rating queries.
Unit 4 - transactions and NoSQL	Atomic booking and concurrency tests; Redis cache-aside / TTL / data structures; Neo4j graph nodes and relationships for recommendations; optional vector search in a later phase.

10.1 Suggested end-to-end acceptance demonstration

1. A student signs in, updates skills and domain interests, and discovers mentors with explainable recommendation reasons.
2. The student submits a request. The mentor accepts it. The system creates one allocation and triggers a notification through the outbox.
3. The mentor publishes a slot. Two test clients attempt to book it concurrently; only one booking is confirmed.
4. The offline session is recorded as completed with topics, skills assessed, observations and next steps.
5. The participants submit structured feedback and the student submits one mentor review for the completed session.
6. The administrator views a request-to-allocation-to-session report and a mentor-rating report.
7. The team demonstrates Redis cache hit/miss/expiry and Neo4j recommendation traversal, then explains why MySQL remains authoritative.

10.2 Reference basis and interpretation

Primary source: Alumni–Student Mentorship Platform: DBMS Project Blueprint (34-page uploaded project blueprint), especially its project vision and roles, modular-monolith architecture, MySQL/Redis/Neo4j responsibilities, conceptual data areas, recommendation design, Redis cache strategy, six business workflows, DBMS-unit mapping and core application priorities.

Requirements that describe business features and architecture are based on that blueprint. The blueprint does not enumerate a formal NFR catalogue or numeric service-level targets; therefore NFR-01 to NFR-14 and the stated initial performance thresholds are proposed product baselines for team review, not quoted source requirements.

PRD completion checkpoint: Before implementation begins, confirm the open decisions in Section 9.4, then use the FR/NFR identifiers as traceable items in the backlog, database design, test cases and project demonstration.